Framework de chiffrement

Travail pratique n° 1 - Programmation libre

Cours SSAD, USTHB, 2026-2027

Chiffrements de César, Hill et Playfair, attaques et protection des mots de passe, authentification des messages

Membres du groupe	Tâche
Membre 1	Chiffrement de César et ses attaques (force brute, fréquences)
Membre 2	Chiffrement de Hill et attaque à clair connu
Membre 3	Chiffrement de Playfair et analyse des digrammes
Membre 4	Attaques sur mots de passe : dictionnaire et force brute, trois cas
Membre 5	Protection des mots de passe et intégrité HMAC (partie 4)
Membre 6	Interface, tests unitaires, mesures de performance, rapport

1. Ce que nous avons réalisé

Le sujet demande un petit framework de chiffrement en quatre parties : une interface, trois chiffrements classiques, une étude des attaques sur les mots de passe avec une solution de protection, et un mécanisme d'authenticité ou d'intégrité. Tout est écrit en Python 3 avec la bibliothèque standard uniquement (Tkinter pour la fenêtre, hashlib et hmac pour le hachage). Le programme se lance avec `python app.py` et les tests avec `python tests.py`.

Fichier	Contenu
ciphers.py	César, Hill, Playfair et les attaques correspondantes
passwords.py	Les trois cas de mots de passe, attaques par dictionnaire et force brute, stockage PBKDF2, verrouillage
integrity.py	Empreinte SHA-256 et tag HMAC, scellement et vérification
app.py	Interface Tkinter à quatre onglets
tests.py	22 tests unitaires et un petit banc d'essai (python tests.py bench)

L'interface comporte quatre onglets. **Messagerie** rejoue le scénario du sujet : l'entité A saisit un message, choisit un algorithme et une clé, puis l'envoie ; le chiffré et son tag d'authentification apparaissent dans une zone « canal » modifiable à la main pour jouer l'attaquant ; l'entité B vérifie le tag puis déchiffre. **Attaques sur les chiffrements** montre ce qu'un attaquant peut faire contre chaque algorithme. **Attaques sur mots de passe** lance les attaques par dictionnaire et force brute sur les trois cas. **Protection** présente les défenses et le tableau des temps.

2. Partie 2 - Chiffrement et déchiffrement

A envoie un message à B. Les deux partagent l'algorithme et la clé. Tout fonctionne sur l'alphabet latin, de A à Z. Pour Hill et Playfair, le texte est d'abord nettoyé : majuscules et lettres seulement (les lettres accentuées sont donc supprimées).

2.1 Chiffrement de César

Chaque lettre est décalée de n positions dans l'alphabet, vers la droite ou vers la gauche. Avec des positions de 0 à 25, la formule est $c = (p + n) \bmod 26$ pour un décalage à droite et $c = (p - n) \bmod 26$ à gauche ; le déchiffrement applique le décalage inverse. La casse et la ponctuation sont conservées. Comme n est pris modulo 26, décaler de 29 revient à décaler de 3.

Failles de sécurité.

- Il n'existe que 26 clés (25 utiles) : tester tous les décalages prend quelques microsecondes. L'interface affiche les 26 candidats et un humain repère tout de suite la bonne ligne.
- Les fréquences des lettres survivent : la lettre la plus fréquente du chiffré est presque sûrement l'image du E, ce qui donne n directement. Nous avons implémenté cette estimation ; elle trouve la clé sur une phrase de quelques dizaines de lettres.
- Les espaces et la ponctuation restent en place, donc la longueur des mots fuit.

2.2 Chiffrement de Hill

La clé est une matrice K de taille n x n à coefficients entiers modulo 26. Le texte est découpé en blocs de n lettres ; chaque bloc est un vecteur colonne p et le bloc chiffré est $c = K\ p \bmod 26$. Le

déchiffrement utilise la matrice inverse, `p = K^-1 c mod 26`. L'inverse n'existe que si pgcd(det K, 26) = 1 ; le programme le vérifie et refuse les mauvaises clés avec un message clair. L'inverse est calculé à partir de la matrice adjointe et de l'inverse modulaire du déterminant (Euclide étendu). Si la longueur du texte n'est pas un multiple de n, on complète avec des X. La clé peut être saisie sous forme de nombres (3,3,2,5) ou d'un mot de 4 ou 9 lettres.

Exemple détaillé, clé [[3,3],[2,5]], message HELP. Les blocs sont HE = (7,4) et LP = (11,15). K(7,4) = (33,34) = (7,8) = HI et K(11,15) = (78,97) = (0,19) = AT, donc le chiffré est HIAT. Cette valeur est vérifiée dans les tests.

Failles de sécurité.

- Le chiffrement est linéaire. Avec n blocs de clair connus et leurs chiffrés, K = C P^-1 mod 26 s'obtient par une seule inversion de matrice. Notre onglet d'attaque le fait pour une clé 2x2 à partir de 4 lettres connues et retrouve la clé exacte.
- Une clé 2x2 n'a que 157 248 matrices inversibles, donc la force brute est facile pour les petites tailles.
- Deux blocs identiques donnent deux chiffrés identiques, ce qui révèle les répétitions (fonctionnement de type ECB).
- Il n'y a pas de diffusion entre blocs : modifier un bloc n'affecte jamais les autres.

2.3 Chiffrement de Playfair

Un carré 5x5 est construit avec le mot clé (lettres répétées supprimées) suivi du reste de l'alphabet, J étant confondu avec I. Le message est découpé en paires ; une lettre double dans une paire reçoit un X intercalé, et une dernière lettre seule est complétée par X. Pour chaque paire : même ligne, on prend la lettre à droite ; même colonne, la lettre en dessous ; sinon on échange les colonnes pour former le rectangle. Le déchiffrement va vers la gauche et vers le haut. Après déchiffrement, le programme retire les X de bourrage (un X entre deux lettres identiques et un X final), pour que B retrouve le texte d'origine ; ce retrait est heuristique, un vrai X à cet endroit serait perdu.

Vérification sur l'exemple classique : clé « playfair example », message « hide the gold in the tree stump » donne BMODZBXDNABEKUDMUIXMMOUVIF, et la suite de tests le contrôle.

Failles de sécurité.

- Il travaille sur des digrammes, soit 600 paires possibles au lieu de 26 lettres, ce qui complique l'analyse de fréquence sans l'empêcher. La fonction de comptage affiche les paires les plus répétées ; sur quelques centaines de lettres elles correspondent aux paires courantes (TH, HE, IN...).
- Une paire et son inverse sont chiffrées en paires inverses (AB -> XY, BA -> YX), une régularité utile à l'attaquant.
- Une lettre n'est jamais appariée à elle-même ni chiffrée en elle-même, ce qui réduit les hypothèses.
- J et I sont confondus et le X de bourrage est ambigu, donc le déchiffrement peut perdre un peu d'information.
- L'escalade de colline (hill climbing) et le recuit simulé le cassent sur quelques centaines de lettres.

3. Partie 3 - Mots de passe

Pour des expériences réalistes, le programme fait ce que permet une base de données volée : il ne garde que l'empreinte SHA-256 du mot de passe, et l'attaquant essaie des candidats, les hache et compare. Trois cas étaient demandés.

Cas	Taille de l'alphabet	Longueur	Espace de recherche
Binaire	2 (0,1)	3	2^3 = 8
Chiffres 1-4	4	4	4^4 = 256
Ensemble a-z A-Z 0-9 + * ...	74	3 (démo)	74^3 = 405224

L'ensemble complet utilisé compte 26 minuscules + 26 majuscules + 10 chiffres + 13 symboles (+ * - _ @ # $! ? % & =), soit 74 caractères. Le sujet dit « etc. », ce choix est donc le nôtre. La longueur du troisième cas est réglable dans l'interface (1 à 4) ; nous prenons 3 dans le rapport car 4 caractères représentent déjà environ 29 millions d'essais.

3.1 Les attaques

Attaque par dictionnaire. L'attaquant hache une liste de mots de passe probables (la nôtre compte une cinquantaine d'entrées : mots de passe courants, courtes suites de chiffres, quelques mots locaux) et compare. Le coût est au plus la taille de la liste, c'est donc la première chose à essayer. Mesure : « qwerty » est trouvé après 4 essais en 7 µs. Un mot de passe aléatoire comme « Xk9+mQ » n'est pas dans la liste et l'attaque s'arrête après 47 essais sans résultat.

Attaque par force brute. Toutes les combinaisons de l'alphabet sont générées avec `itertools.product` et testées. Elle réussit toujours, seule la durée change.

Temps minimal. Pour chaque cas, la stratégie combinée est : dictionnaire d'abord (peu coûteux, couvre les choix humains faibles), puis recherche exhaustive sur l'alphabet exact et la longueur exacte, ce qui donne le plus petit espace possible. Pour les deux petits cas, l'espace entier est plus petit que le coût du dictionnaire, la recherche exhaustive est donc instantanée. Dans la boucle interne, nous comparons aussi les octets bruts de l'empreinte plutôt que des chaînes hexadécimales, ce qui est plus rapide.

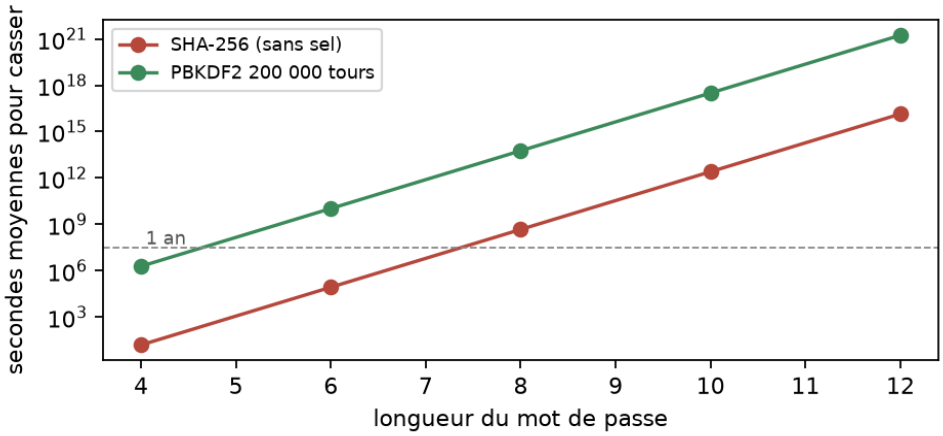
Cas	Espace	Essais (pire cas)	Temps
3 car., 0/1	8	8	36 µs
4 car., 1-4	256	256	301 µs
3 car., ensemble complet	405224	405224	426.9 ms

La machine utilisée atteint environ 970762 essais SHA-256 par seconde en Python (une carte graphique en fait des milliards par seconde, les vrais attaquants sont donc des milliers de fois plus rapides).

3.2 Protection

Les attaques ci-dessus réussissent parce que le hachage est rapide et que l'alphabet ou la longueur sont petits. Notre proposition a quatre couches, toutes implémentées dans passwords.py.

- **Hachage salé et lent.** Les mots de passe sont stockés avec PBKDF2-HMAC-SHA256, un sel aléatoire de 16 octets par utilisateur et 200 000 itérations. Le sel neutralise les tables précalculées et fait que deux utilisateurs ayant le même mot de passe ont des empreintes différentes. Le nombre d'itérations rend chaque essai coûteux : une vérification coûte environ 127.8 ms ici, l'attaquant passe donc d'environ 970762 essais par seconde à environ 7, soit un ralentissement d'environ 124101x.
- **Limitation de débit et verrouillage.** La classe LoginGuard verrouille le compte 30 secondes après 3 échecs. Un attaquant en ligne n'a alors que 6 essais par minute au lieu de milliers. Les tests vérifient qu'un bon mot de passe est refusé pendant le verrouillage.
- **Politique de mot de passe.** strength_problems refuse les mots de passe de moins de 10 caractères, ceux de la liste courante, et ceux sans minuscule, majuscule, chiffre ou symbole. Cela neutralise le dictionnaire et fait croître exponentiellement l'espace de la force brute.
- **Comparaison en temps constant.** hmac.compare_digest est utilisé pour que le temps de réponse ne révèle pas combien d'octets correspondent.



Le graphique montre le temps moyen pour casser un mot de passe de l'ensemble de 74 caractères selon sa longueur. Sans sel et avec un hachage rapide, 8 caractères tiennent environ 14.7 ans sur notre matériel ; avec le hachage lent, le même mot de passe tient 1.8e+06 ans. Chaque caractère ajouté multiplie le travail par 74.

Remarque importante : les deux petits cas (8 et 256 combinaisons) ne peuvent pas être protégés par le hachage, car n'importe quel attaquant peut tout essayer instantanément même avec un hachage lent (256 x 0,14 s font environ 36 secondes). La seule défense est le verrouillage et, bien sûr, ne pas utiliser de tels mots de passe.

4. Partie 4 - Authenticité et intégrité

Nous avons choisi un objectif : **permettre à B de détecter toute modification du message et de savoir qu'il vient de quelqu'un qui possède le secret partagé.** L'outil est un tag HMAC-SHA256 calculé sur le chiffré avec un secret partagé par A et B (chiffrer puis authentifier).

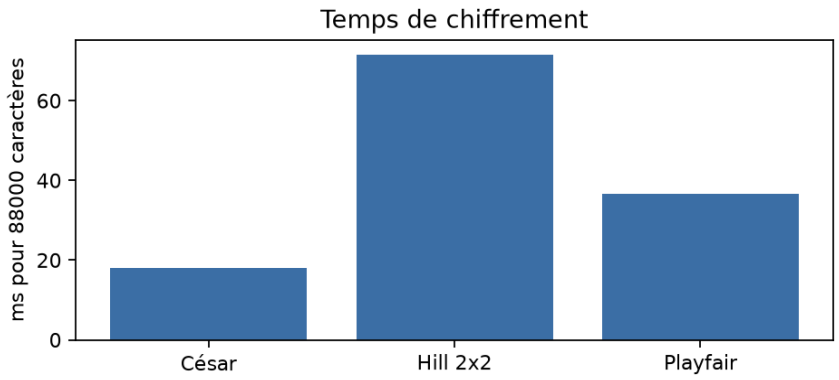
- A calcule tag = HMAC(secret, chiffré) et envoie les deux.
- B recalcule le tag sur ce qu'il a reçu et compare en temps constant. S'il diffère, le message est rejeté avant tout déchiffrement.

- Dans l'onglet Messagerie, le bouton « Changer une lettre » simule un attaquant qui modifie une lettre sur le canal ; B affiche alors « intégrité non vérifiée ». Un mauvais secret donne le même résultat, ce qui illustre l'authenticité.

Sans cette couche, les chiffrements classiques n'offrent aucune intégrité : en César, changer une lettre du chiffré modifie silencieusement la lettre correspondante du clair. Notons aussi qu'un HMAC prouve que l'émetteur connaît le secret mais n'apporte pas la non-répudiation ; pour cela il faudrait des signatures numériques.

5. Tests et performances

Les 22 tests unitaires de tests.py réussissent tous. Ils couvrent : les allers-retours de César dans les deux sens pour des décalages de 0 à 59, les attaques connues sur César, l'exemple classique de Hill (clé GYBNQKURP, ACT -> POH), 40 clés Hill aléatoires de taille 2 et 3, le rejet d'une clé non inversible, l'attaque à clair connu, le vecteur de référence de Playfair, les messages de longueur impaire, les tailles d'espaces de recherche, la force brute sur chacun des 8 mots de passe binaires, le succès et l'échec du dictionnaire, la vérification PBKDF2 et l'unicité du sel, le temps de verrouillage, et la détection de modification par HMAC.



Algorithme	Temps pour 88000 caractères
César	18.0 ms
Hill 2x2	71.6 ms
Playfair	36.6 ms

Les trois chiffrements traitent environ 90 ko de texte en moins d'un dixième de seconde, la vitesse n'est donc pas un problème. Hill est le plus lent par caractère à cause des produits de matrices, César le plus rapide. La vraie différence entre eux n'est pas la vitesse mais la résistance, vue en section 2.

6. Répartition du travail

Membre	Fichiers / sections	Livrable
1	ciphers.py (caesar, caesar_bruteforce, caesar_frequency_guess)	César dans les deux sens, attaques, section 2.1
2	ciphers.py (fonctions Hill, inverse de matrice mod 26)	Chiffrement de Hill, attaque à clair connu, section 2.2
3	ciphers.py (fonctions Playfair, digram_counts)	Playfair, retrait du bourrage, section 2.3
4	passwords.py (espace de recherche, dictionnaire, force brute)	Tests binaires, mesures de temps, section 3.1

5	passwords.py (PBKDF2, LoginGuard), integrity.py	Protection, HMAC, sections 3.2 et 4
6	app.py, tests.py, rapport	Interface, 22 tests, mesures, sections 1 et 5

7. Conclusion

Les trois chiffrements fonctionnent, font l'aller-retour correctement et sont vérifiés sur des exemples publiés, mais chacun tombe devant une attaque connue que notre interface démontre : recherche exhaustive de clé ou fréquences pour César, clair connu pour Hill, statistiques de digrammes pour Playfair. Ils servent à apprendre, pas à protéger de vraies données. Pour les mots de passe, les petits alphabets et les longueurs courtes sont cassés instantanément quoi que l'on fasse, et les plus grands ne sont sûrs qu'avec salage, hachage lent, politique de mots de passe et verrouillage. Enfin, la couche HMAC montre que la confidentialité seule ne suffit pas : l'intégrité doit être vérifiée séparément.